

De SQL para MongoDB

Traduzindo o que você já sabe

Unidade 7 — NoSQL · Banco de Dados · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Terminologia equivalente

SQL (relacional)	MongoDB (documento)
Database	Database
Tabela	Coleção (collection)
Linha / registro	Documento (JSON/BSON)
Coluna / atributo	Campo (field)
Chave primária	campo <code>_id</code>
JOIN	<code>\$lookup</code> ou dado embutido
Índice	Índice

Tabela × Documento

Relacional (normalizado)

```
clientes(id, nome)
pedidos(id, cliente_id, data)
itens(pedido_id, produto, qtd)
```

Dados espalhados em 3 tabelas → JOINS para reunir.

Documento (agregado)

```
{ "_id": 1, "nome": "Ana",
  "pedidos": [
    { "data": "2026-03-01",
      "itens": [
        { "produto": "Mouse", "qtd": 2 }
      ]
    }
  ]
}
```

O pedido inteiro em **um documento**.

Criar e evoluir estrutura

SQL

```
CREATE TABLE people (  
  id      NUMBER PRIMARY KEY,  
  user_id VARCHAR2(30),  
  status  VARCHAR2(1)  
);  
ALTER TABLE people  
  ADD join_date DATE;
```

MongoDB

```
// coleção criada ao inserir  
db.people.insertOne({  
  user_id: "abc", status: "A"  
})  
// "adicionar coluna" = só gravar o campo  
db.people.updateOne(  
  { user_id: "abc" },  
  { $set: { join_date: new Date() } }  
)
```

Inserir

SQL

```
INSERT INTO people  
  (user_id, age, status)  
VALUES ('bcd001', 45, 'A');
```

MongoDB

```
db.people.insertOne({  
  user_id: "bcd001",  
  age: 45,  
  status: "A"  
})
```

Consultar (SELECT → find)

SQL

```
SELECT user_id, status
FROM people
WHERE status = 'A'
      AND age > 25
ORDER BY user_id;
```

MongoDB

```
db.people.find(
  { status: "A", age: { $gt: 25 } },
  { user_id: 1, status: 1, _id: 0 }
).sort({ user_id: 1 })
```



LIKE '%bc%' → { user_id: /bc/ } (expressão regular).

Atualizar e Excluir

SQL

```
UPDATE people
SET status = 'C'
WHERE age > 25;

DELETE FROM people
WHERE status = 'D';
```

MongoDB

```
db.people.updateMany(
  { age: { $gt: 25 } },
  { $set: { status: "C" } }
)

db.people.deleteMany(
  { status: "D" }
)
```

Agregação (GROUP BY → aggregate)

SQL

```
SELECT status,  
       COUNT(*) qtd,  
       AVG(age) media  
FROM people  
GROUP BY status  
HAVING COUNT(*) > 1;
```

MongoDB

```
db.people.aggregate([  
  { $group: {  
    _id: "$status",  
    qtd: { $sum: 1 },  
    media: { $avg: "$age" } } },  
  { $match: { qtd: { $gt: 1 } } }  
])
```


Relacionamentos: embutir × referenciar

- **Embutir** (documento aninhado): dados lidos **sempre juntos**, cardinalidade limitada → 1 leitura, sem JOIN.
- **Referenciar** (`_id` de outra coleção + `$lookup`): dados grandes/compartilhados, relação N:N.

```
db.pedidos.aggregate([
  { $lookup: {
    from: "clientes", localField: "cliente_id",
    foreignField: "_id", as: "cliente" } }
])
```

Regra de ouro: **modele pelos padrões de acesso**. "O que é lido junto, guarde junto."

Resumo

- Você **já sabe** os conceitos — muda a **forma** de guardar e consultar.
- MongoDB troca **JOINS** por **agregados** e favorece **flexibilidade + escala**.
- Sintaxe atual: `insertOne/Many`, `updateOne/Many`, `deleteOne/Many`, `countDocuments`, `aggregate`.
- Pratique na **VM** (container Docker) e leve para o **Projeto Integrador** (inclusive na nuvem).

Dúvidas?

howard.cruz@faesa.br